Web & Backend Basics

A practical beginner guide to Client–Server Architecture, HTTP/HTTPS, APIs, Status Codes, Nginx, Proxies, VPN, Hashing and Encryption

Purpose: This document is a revision guide for the concepts covered in class. The examples are intentionally practical so that the student can connect each concept with something they see while using websites and applications.

1. Client–Server Architecture

Basic idea: A client is the part that requests a service. A server is the system that receives the request, processes it, and sends a response.

Client can be a browser, mobile app, desktop app, or another program. Server usually runs backend code, talks to databases, performs business logic, and returns data.

Simple flow: Client → Request → Server → Processing → Response → Client

> Real-life example: Think of a restaurant. You are the customer (client), the waiter carries your request, the kitchen is the server doing the work, and the waiter brings the result back to you.

> Website example: When you open an online shopping website, your browser requests product information. The server may fetch products from a database and send the product data back to the browser.

A useful point for students: the browser normally should not directly access the database. The backend acts as the controlled middle layer.

2. How Does a Browser Communicate With a Server?

The browser and server communicate over a network using protocols. For normal web applications, HTTP or HTTPS is used.

Basic sequence:

- You type a website address or click a link.
- The browser finds the server's network address and opens a connection.
- The browser sends an HTTP/HTTPS request.
- The server receives the request and runs the required backend logic.
- The server sends an HTTP/HTTPS response.
- The browser reads the response and displays the result.

> Real-life example: You open a student portal and click “My Attendance”. The browser sends a request such as GET /attendance. The server checks the logged-in user, reads attendance data, and returns the result.

Important: A URL tells the client where the resource is. HTTP/HTTPS defines how the request and response are communicated.

3. HTTP — HyperText Transfer Protocol

HTTP is a protocol used for communication between clients and web servers. It defines the format and rules for requests and responses.

```
Typical request:
```

```
GET /students/101 HTTP/1.1
Host: example.com
```

Typical response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"name":"Rahul","attendance":86}
```

Simple example: A browser asks, “Give me Rahul's attendance.” The server replies, “Here is the attendance data.”

4. HTTPS — Secure HTTP

HTTPS is HTTP protected using TLS (Transport Layer Security). It is used to protect data while it travels between the client and server.

What does HTTPS provide?

- Encryption: helps prevent outsiders on the network from reading the exchanged data.
- Integrity: helps detect unauthorized modification of data in transit.
- Authentication: TLS certificates help the browser verify that it is communicating with the intended website.

Example: If you submit a password on a properly configured HTTPS website, the password is not sent across the network as plain readable text. TLS protects the connection.

Important distinction: HTTPS protects data in transit. It does not automatically make a weak password, insecure backend, or badly designed application safe.

5. What Is an API?

API stands for Application Programming Interface. In a web application, an API commonly provides endpoints through which a client can request data or ask the backend to perform an operation.

Example: A frontend needs the list of students. Instead of knowing how the database works, it calls an endpoint such as:

```
GET /api/students
```

The backend handles the database query and returns structured data, often JSON.

Real-life example: A waiter can be compared to an API. You do not enter the kitchen and cook your own food. You give the waiter a request in an expected format; the restaurant processes it and returns the result.

6. Very Basic API Example Using Node.js

Node.js can be used to create a backend server. A framework such as Express makes it easier to define API routes.

A minimal example:

```
const express = require('express');
const app = express();

app.get('/api/hello', (req, res) => {
  res.json({ message: 'Hello World' });
});

app.listen(3000);
```

If the server is running on port 3000, a client can request GET /api/hello and receive JSON such as { "message": "Hello World" }.

Key idea: Route → Backend logic → Response.

7. HTTP Status Codes

Status codes are numbers sent by the server to describe the result of a request. The first digit gives a broad category.

Code	Meaning	Simple example
200	OK	Student data returned successfully.
201	Created	A new student/account was created.
400	Bad Request	Required field is missing or request format is invalid.
401	Unauthorized	Authentication is missing or invalid.
403	Forbidden	User is authenticated but does not have permission.
404	Not Found	Requested API route or resource does not exist.
500	Internal Server Error	Unexpected error occurred on the server.

Flow example: A student opens the profile page → browser calls GET /api/profile → server checks login → database is queried → server returns 200 with profile data.

401 vs 403: 401 generally means “you are not properly authenticated”; 403 means “you are authenticated, but you are not allowed to do this.”

8. What Is Nginx?

Nginx is a high-performance web server and reverse proxy. It can receive traffic from clients and route that traffic to the appropriate backend service.

Example deployment:

```
Browser → HTTPS :443 → Nginx → Node.js :3000
```

The browser does not need to know that Node.js is listening on port 3000. Nginx receives the public request and forwards it to the internal application.

Real-life example: Think of Nginx as the security/reception desk of a large office building. Visitors enter through one reception area, and the receptionist directs each visitor to the correct department.

9. Reverse Proxy vs Forward Proxy

Type	Who it represents?	Simple use
Forward Proxy	Client	Client sends traffic through a proxy before reaching the internet.
Reverse Proxy	Server	Clients connect to the reverse proxy; it forwards requests to backend servers.

Forward proxy: `Client → Forward Proxy → Internet`

Reverse proxy: `Client → Nginx → Backend Server`

Why use a reverse proxy? Common reasons include routing multiple applications, TLS termination, load balancing, hiding internal service ports, serving static files, and adding request controls.

10. What Is a VPN?

VPN stands for Virtual Private Network. A VPN creates an encrypted connection between a device and a VPN service/network, allowing traffic to travel through that connection.

Simple picture: `Device → Encrypted VPN Tunnel → VPN Server → Destination`

Real-life example: Suppose you are using public Wi-Fi at a café. A VPN can create an encrypted tunnel between your device and the VPN server, reducing exposure of traffic between your device and the VPN server.

Important: A VPN and Nginx reverse proxy solve different problems. A VPN is mainly about secure/private network connectivity; a reverse proxy is about receiving and routing application traffic on behalf of servers.

11. What Is Hashing?

Hashing converts input data into a fixed-length output called a hash. A secure cryptographic hash is designed to be one-way in practical use: you normally do not decrypt a hash to recover the original input.

Example: A password should not normally be stored in a database as plain text. A password hashing algorithm such as Argon2id or bcrypt can be used to store a password verifier instead.

Important: Hashing is not encryption. There is no “decrypt hash” operation for a properly designed password hash.

Example idea:

```
password → password hashing algorithm → stored hash
login password → same verification process → compare with stored verifier
```

12. What Is Encryption?

Encryption transforms readable data (plaintext) into protected data (ciphertext) using an encryption method and key. With the appropriate key, authorized parties can decrypt the ciphertext.

Simple idea:

```
Plaintext + Key → Encryption → Ciphertext
Ciphertext + Key/appropriate key → Decryption → Plaintext
```

Real-life example: HTTPS uses cryptographic mechanisms to protect communication between the browser and server. The goal is that someone observing the network traffic cannot simply read the protected application data.

13. Hashing vs Encryption — Easy Comparison

Feature	Hashing	Encryption
Main purpose	Integrity / password verification	Confidentiality
Normally reversible?	No	Yes, with the appropriate key
Uses a key?	Cryptographic hashes do not work like key-based encryption	Yes
Example	Password storage with bcrypt/Argon2id	Protecting data in a secure communication channel

Memory trick: Hashing is like making a fingerprint of data; encryption is like putting data inside a locked box.

14. Putting Everything Together

Imagine a student attendance website:

```
Student Browser → HTTPS → Nginx → Node.js API → Database
```

- The browser is the client.
- Node.js backend is the server/application.
- The API is the communication interface between frontend and backend.

- HTTPS protects communication in transit.
- Nginx acts as the reverse proxy in front of the application.
- The backend may return status codes such as 200, 401, 403, 404 or 500.
- Passwords should be hashed, not stored as plain text.
- Sensitive data that needs to be recovered should use appropriate encryption, not hashing.

Final real-life analogy: Think of a college office. The student is the client, the reception desk is Nginx, the staff member handling the request is the backend, the official request form is the API contract, the database is the record room, and HTTPS is the secure way the information travels between the student and the office.

15. Quick Revision Questions

1. What is the difference between a client and a server?
2. What happens after a browser sends an HTTP request?
3. Why is HTTPS preferred over plain HTTP for sensitive communication?
4. What is an API endpoint?
5. What is the difference between 401 and 403?
6. What is the job of Nginx as a reverse proxy?
7. What is the difference between a forward proxy and a reverse proxy?
8. What problem does a VPN solve?
9. Why are passwords hashed instead of encrypted for normal password storage?
10. What is the main difference between hashing and encryption?

One-line summary: The browser acts as a client, the server processes requests, APIs define how applications communicate, HTTP/HTTPS carries web communication, Nginx can sit in front as a reverse proxy, status codes describe request results, VPN provides a protected network tunnel, hashing creates a one-way verifier, and encryption protects data so authorized parties can recover it.